Department of Information Engineering Technology

UNEVOC Network Member

Advance Database Management System
Fall 2025 , BS(CS)

Dr. HUMA ISRAR

DEPARTMENT OF COMPUTING

NATIONAL SKILLS UNIVERSITY, ISLAMABAD, PAKISTAN

Lecture 6

Databases Recovery Outline

- Purpose of Database Recovery
- Types of Failure
- Transaction Log
- Data Updates and Data Caching
- Transaction Roll-back (Undo) and Roll-Forward
- Checkpointing
- Recovery schemes

Database Recovery

- A Purpose of Database Recovery
  - To bring the database into the last consistent state, which existed prior to the failure.
  - To preserve transaction properties (Atomicity, Consistency, Isolation and Durability).
- Example:
- If the system crashes before a fund transfer transaction completes its execution, then either one or both accounts may have incorrect value. Thus, the database must be restored to the state before the transaction modified any of the accounts.

Recovery Concepts

- Recovery process restores database to most recent consistent state before time of failure
- Information kept in system log
- Typical recovery strategies
- Restore backed-up copy of database
  - Best in cases of extensive damage
- Identify any changes that may cause inconsistency
  - Best in cases of noncatastrophic failure
  - Some operations may require redo

Types of Failure

- The database may become unavailable for use due to
- Transaction failure: Transactions may fail because of incorrect input, deadlock, incorrect synchronization.
- System failure: System may fail because of addressing error, application error, operating system fault, RAM failure, etc.
- Media failure: Disk head crash, power disruption, etc.

Recovery Algorithms

- A Recovery algorithms have two parts
- Actions taken during normal transaction processing to ensure that enough information exists to recover from failures
- Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

Transaction Log

- For recovery from any type of failure data values prior to modification (BFIM - BeFore Image) and the new value after modification (AFIM – AFter Image) are required.

- These values and other information is stored in a sequential file called Transaction log. A sample log is given below. Back P and Next P point to the previous and next log records of the same transaction.

Transaction Log

- BFIM (Before Image):
  - The old value of the data item (before update).
- AFIM (After Image):
  - The new value of the data item (after update).
- These values help in two types of recovery:
- UNDO: When a transaction fails or crashes before commit
  - Use BFIM to restore the old value.
- REDO: When a transaction commits but values are not written to disk
  - Use AFIM to re-apply the change

Recovery

- State (BFIM - BeFore Image) and (AFIM – AFter Image)

Back P and Next P make this efficient by creating a linked list of each transaction's actions inside the log.

T ID	Back P	Next P	Operation	Data item	BFIM	AFIM
T1	0	1	Begin			
T1	1	4	Write	X	X = 100	X = 200
T2	0	8	Begin			
T1	2	5	W	Y	Y = 50	Y = 100
T1	4	7	R	M	M = 200	M = 200
T3	0	9	R	N	N = 400	N = 400
T1	5	nil	End			

Data Update

- Immediate Update: As soon as a data item is modified in cache, the disk copy is updated.
- Deferred Update: All modified data items in the cache is written either after a transaction ends its execution or after a fixed number of transactions have completed their execution.
- Shadow update: The modified version of a data item does not overwrite its disk copy but is written at a separate disk location.
- In-place update: The disk version of the data item is overwritten by the cache version.

Data Caching

- Data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk.
- (Data cahche is RAM area managed by cache manager)
- The flushing is controlled by Modified and Pin-Unpin bits.
- Pin bit = “Don’t write to disk yet.”
- Unpin bit = “Now you can write to disk.”
- Modified bit = “This value has changed and must be written to disk eventually.

Transaction operation

- Transaction Roll-back (Undo) and Roll-Forward (Redo)
- To maintain atomicity, a transaction's operations are redone or undone.
- Undo: Restore all BFIMs on to disk (Remove all AFIMs).
- Redo: Restore all AFIMs on to disk.
- Database recovery is achieved either by performing only Undos or only Redos or by a combination of the two. These operations are recorded in the log as they happen.

Database Recovery

(a)

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

T_3
read_item(C)
write_item(B)
read_item(A)
write_item(A)

Figure 19.1
Illustrating cascading rollback (a process that never occurs in strict or cascadeless schedules). (a) The read and write operations of three transactions. (b) System log at point of crash. (c) Operations before the crash.

Three sample transactions

(b)

		A	B	C	D
		30	15	40	20
	[start_transaction,T_3]				
	[read_item,T_3,C]				
*	[write_item,T_3,B,15,12]		12		
	[start_transaction,T_2]				
	[read_item,T_2,B]				
**	[write_item,T_2,B,12,18]		18		
	[start_transaction,T_1]				
	[read_item,T_1,A]				
	[read_item,T_1,D]				
	[write_item,T_1,D,20,25]				25
	[read_item,T_2,D]				
**	[write_item,T_2,D,25,26]				26
	[read_item,T_3,A]				

← System crash

Figure 19.1
Illustrating cascading rollback (a process that never occurs in strict or cascadeless schedules). (a) The read and write operations of three transactions. (b) System log at point of crash. (c) Operations before the crash.

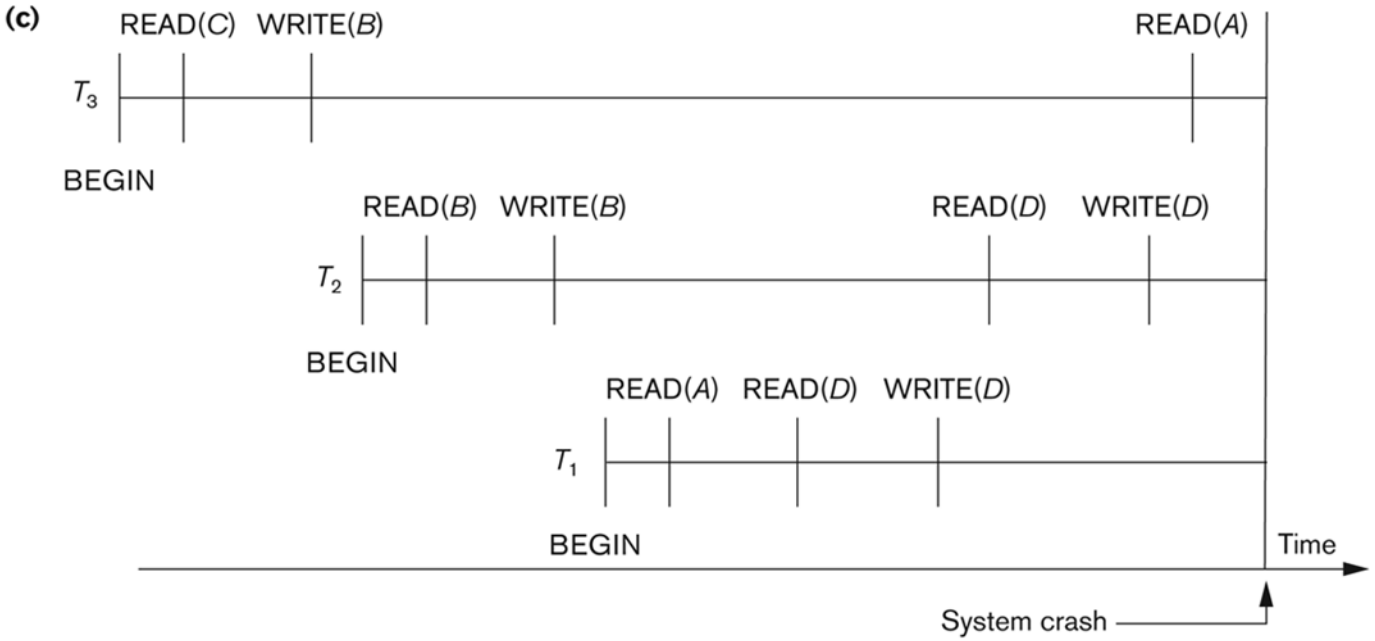
* T_3 is rolled back because it did not reach its commit point.

** T_2 is rolled back because it reads the value of item B written by T_3.

Database Recovery

Roll-back: One execution of T1, T2 and T3 as recorded in the log.

Figure 19.1
Illustrating cascading rollback (a process that never occurs in strict or cascadeless schedules). (a) The read and write operations of three transactions. (b) System log at point of crash. (c) Operations before the crash.



Steal vs No-Steal Policy

- When Database allows the Cache Manager to write (flush) a modified page to disk even before the transaction commits**.
- Why “steal”?
- Because DBMS is stealing an uncommitted data page from a transaction and writing it to disk early.
- No-Steal Policy
- DBMS does NOT allow writing any modified (dirty) page to disk until the transaction commits.

Force vs No-Force Policy

- Force Policy
- When a transaction commits, all modified pages MUST be written to disk immediately
- No-Force Policy
- At commit time, DBMS does not force pages to disk.
- The updated data stays in cache and may be written later

WAL is the most important rule in recovery systems.

Write-Ahead Logging

- When in-place update (immediate or deferred) is used then log is necessary for recovery and it must be available to recovery manager. This is achieved by Write-Ahead Logging (WAL) protocol. WAL states that
- For Undo: Before a data item's AFIM is flushed to the database disk (overwriting the BFIM) its BFIM must be written to the log and the log must be saved on a stable store (log disk).
- For Redo: Before a transaction executes its commit operation, all its AFIMs must be written to the log and the log must be saved on a stable store.

Check-pointing

Time to time (randomly or under some criteria) the database flushes its buffer to database disk to minimize the task of recovery. The following steps defines a checkpoint operation:

1. Suspend execution of transactions temporarily.
2. Force write modified buffer data to disk.
3. Write a [checkpoint] record to the log, save the log to disk.
4. Resume normal transaction execution.

During recovery redo or undo is required to transactions appearing after [checkpoint] record.

Any Question ??